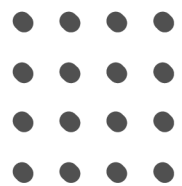




VMES Grinds





?

Graph Fundamentals & BFS/DFS



Graph Fundamentals

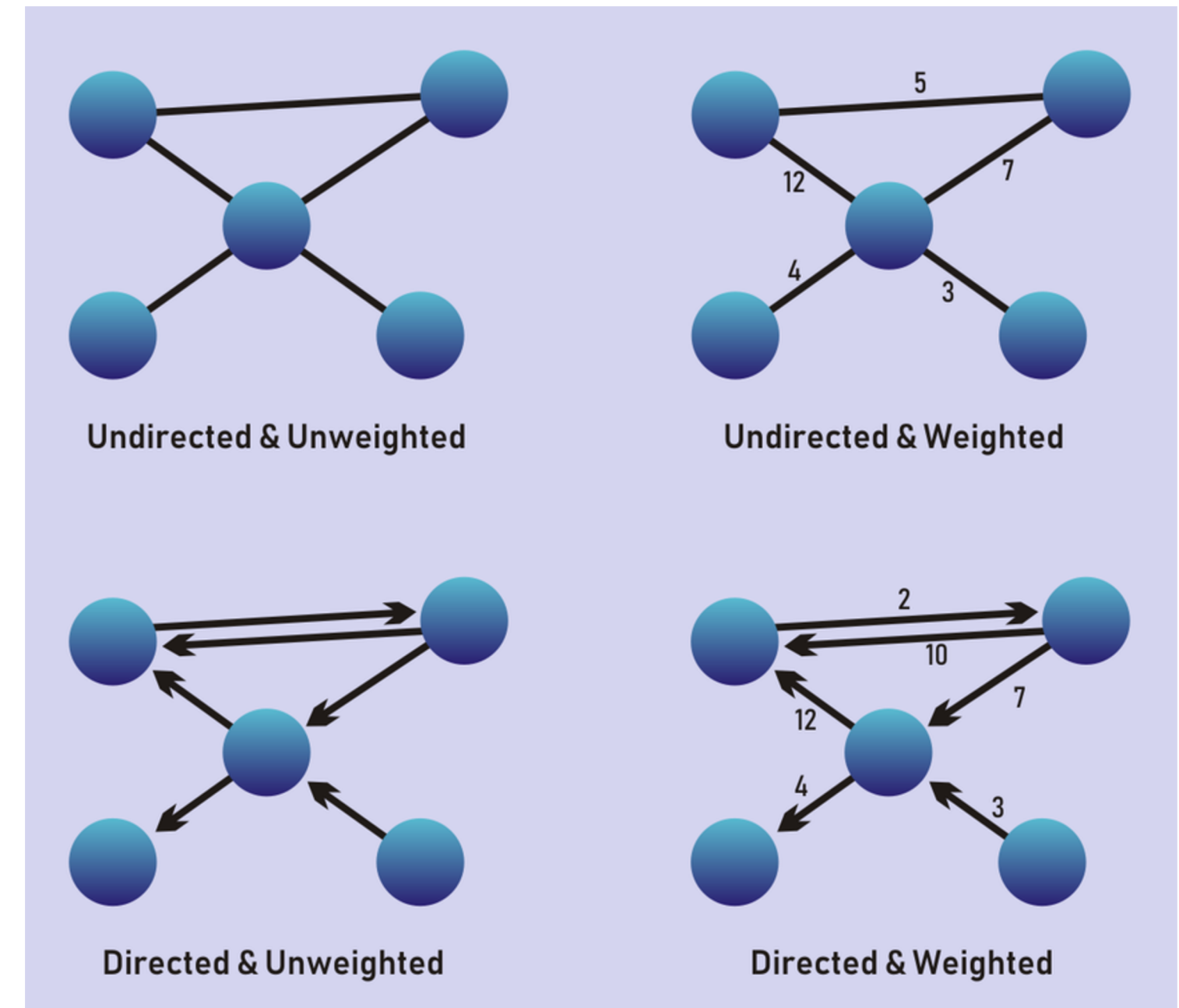
Definition

A Graph $G = (V, E)$ consists of:

- V : A set of Vertices (or Nodes).
- E : A set of Edges (links between nodes).

Graph Types

- **Directed (Digraph)**: Edges have direction ($u \rightarrow v$).
 - *Ex: Web links, Twitter followers.*
- **Undirected**: Edges are bidirectional ($u \leftrightarrow v$).
 - *Ex: Road networks, Facebook friends.*
- **Weighted**: Edges carry a value (cost/distance).
 - *Ex: Distance between cities.*



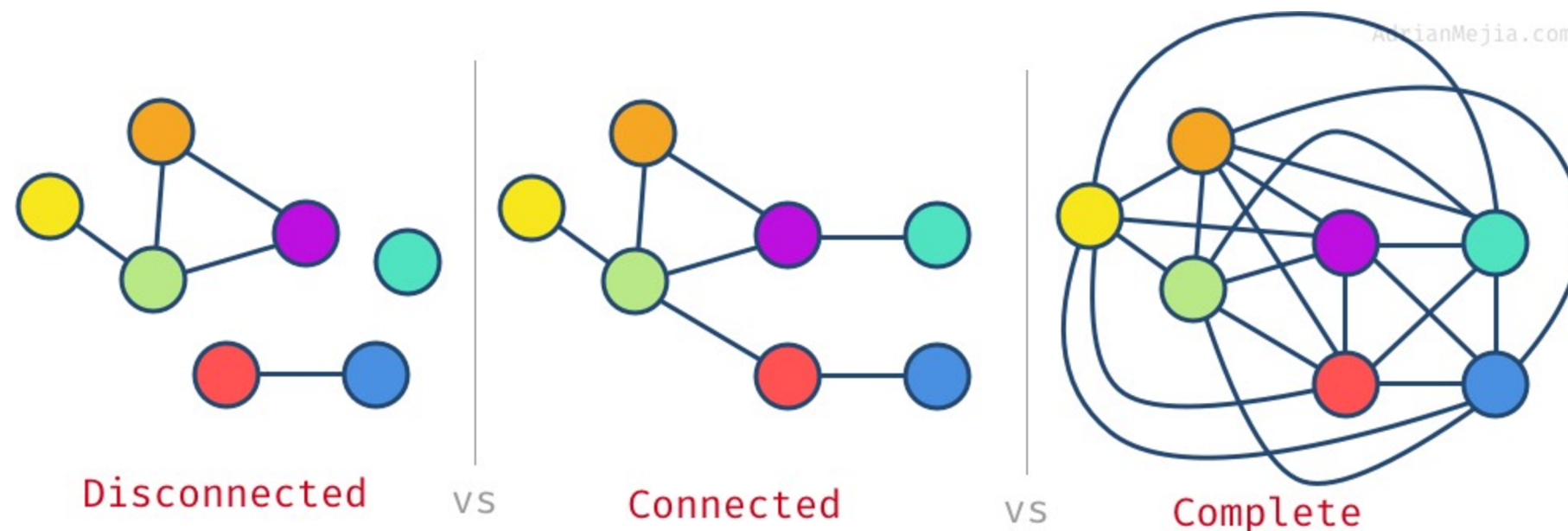
- Symmetric relationships
- One-way relationships
- Edges carry a value



Essential Terminology

Structure Properties

- **Path:** A sequence of vertices connected by edges.
- **Cycle:** A path that starts and ends at the same vertex.
- **Degree:** Number of edges connected to a vertex.
 - In-degree: Incoming edges.
 - Out-degree: Outgoing edges.



Connectivity Types

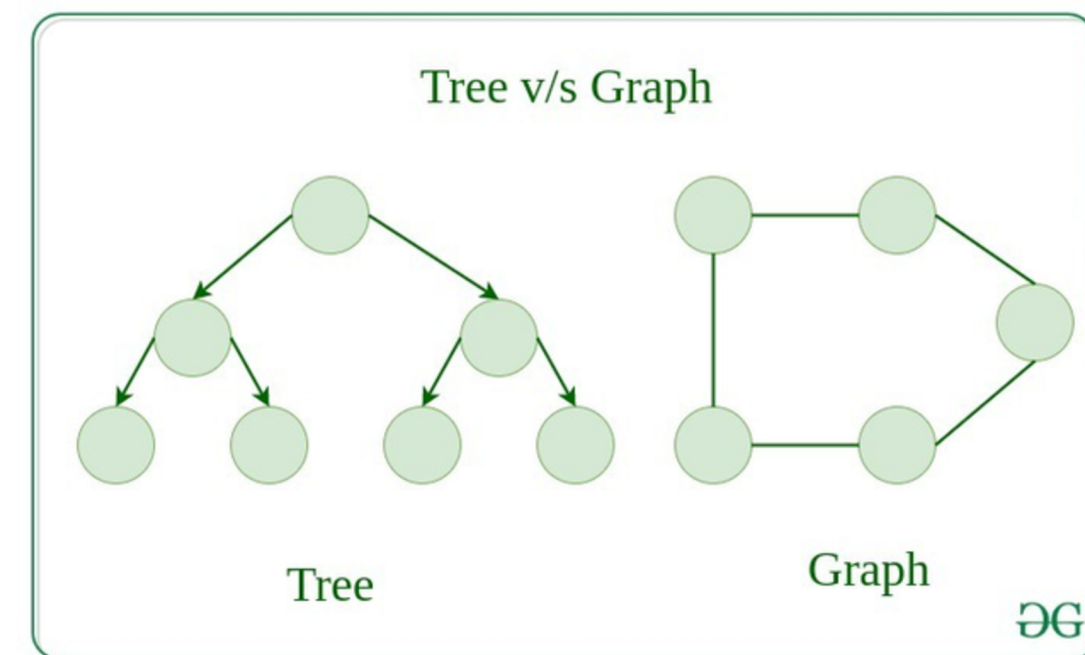
Connected Graph:

- There is a path between every pair of vertices

Tree:

- A connected graph with no cycles.
- Property: $|E| = |V| - 1$

Forest: A collection of disjoint trees.





Graph Representations

1. Adjacency Matrix

A 2D array $A[V][V]$ where $A[i][j] = 1$ if edge exists, else 0.

- Space: $O(V^2)$
- Check Edge: $O(1)$
- Good for: Dense graphs

2. Adjacency List

An array of lists. Index i stores neighbors of vertex i .

- Space: $O(V + E)$
- Check Edge: $O(\text{degree}(u))$
- Good for: Sparse graphs (most real-world graphs)

○

	0	1	2
0	0	1	1
1	1	0	0
2	1	0	0

Matrix: fast lookup, high memory

```
0: [1, 2]
1: [0]
2: [0]
```

List: efficient memory



Breadth-First Search (BFS)

Concept

- Explores the graph layer by layer using a Queue (FIFO)
 - Start at source s .
 - Visit all neighbors of distance k before visiting neighbors at distance $k+1$

Key Properties

Data Structure: Queue

Pathfinding: Finds the shortest path in unweighted graphs.

Time Complexity: $O(V + E)$

BFS Algorithm States

- WHITE: Undiscovered
- GRAY: Discovered (in Queue)
- BLACK: Finished (Processed)
 - $d[v]$: Distance from source to v
 - $p[v]$: Parent of v in BFS tree



BFS Implementation (Python)

```
def BFS(G, s):
    global d, p, color
    Q = [s]          # Initialize Queue
    d[s] = 0
    color[s] = "GRAY"

    while Q:
        u = Q.pop(0) # Dequeue

        for v in G[u]:
            if color[v] == "WHITE":
                color[v] = "GRAY"
                d[v] = d[u] + 1
                p[v] = u
                Q.append(v) # Enqueue

        color[u] = "BLACK" # Finished
```

Core steps

1. Enqueue start node.
2. While Queue not empty:
3. Dequeue u.
4. Check neighbors v.
5. If WHITE: Update distance, parent, set GRAY, Enqueue.
6. Set u to BLACK.



BFS Algorithm Visualizer



Graph Input

```
5 6
1 2
1 3
2 4
2 5
3 4
```

Start Node:

1

Visualize Graph

Interactive BFS Algorithm Visualizer & Step-by-Step Guide

Visualize breadth-first search in real-time with interactive controls, queue tracking, and node state monitoring. Learn BFS algorithm fundamentals with step-by-step execution.



Depth-First Search (DFS)

Concept

- Explores as deep as possible along each branch before backtracking using a Stack (LIFO) or Recursion.
- Visit child, then child's child, etc.
- Backtrack when no unvisited neighbors remain.

Key Properties

Data Structure: Stack / Recursion

Timestamps: Tracks discovery time (d) and finish time (f).

Time Complexity: $O(V + E)$

DFS Algorithm States

- **d[v] (Discovery Time):** Time when v becomes GRAY (first visited).
- **f[v] (Finish Time):** Time when v becomes BLACK (all neighbors processed).

Useful for topological sorting and strong connectivity.



BFS Implementation (Python)

```
def DFS_Visit(u, G):
    global time, color, d, f, p
    time += 1
    d[u] = time
    color[u] = "GRAY"

    for v in G[u]:
        if color[v] == "WHITE":
            p[v] = u
            DFS_Visit(v, G)

    color[u] = "BLACK"
    time += 1
    f[u] = time

def DFS(G):
    for u in range(len(G)):
        if color[u] == "WHITE":
            DFS_Visit(u, G)
```

Core steps

1. Mark u as GRAY.
2. Record discovery time.
3. Recursively visit all WHITE neighbors.
4. Mark u as BLACK.
5. Record finish time.



BY ANTHROPIC

Interactive DFS Algorithm Visualizer & Step-by-Step Guide

Visualize depth-first search algorithms in real-time with interactive step-by-step execution, node state tracking, and edge classification for learning graph theory.



Key Concepts & Application

Connected Components

Use **traversal** (BFS or DFS) to group connected vertices.

Run traversal.

If unvisited nodes remain, increment component count and start new traversal from unvisited node.

Cycle Detection

- **Undirected:** During traversal, if we see a visited neighbor that is NOT parent $\rightarrow$ Cycle.
- **Directed:** During DFS, if we see a Back Edge (edge to a GRAY ancestor) $\rightarrow$ Cycle.

○

Topological Sort (DAG Only)

Linear ordering of vertices such that for every edge $u \rightarrow v$, u comes before v .

Method: Run DFS. Sort vertices by Finish Time ($f[v]$) in descending order.

Used for scheduling tasks with dependencies.



Comparison: BFS vs DFS

Feature	BFS (Breadth-First)	DFS (Depth-First)
Data Structure	Queue (FIFO)	Stack (LIFO) / Recursion
Traversal Shape	Radiates out (Circular)	Deep path then backtrack
Shortest Path	Guaranteed (unweighted)	Not Guaranteed
Cycle Detection	Possible	Excellent (Back edges)
Best Use Case	Shortest Path, Peer-to-Peer Networks	Mazes, Topological Sort, Connectivity